University of Asia Pacific
Department of Computer Science and Engineering

Tic Tac Toe Game using the Minimax Algorithm with Alpha-Beta Pruning

An Adversarial Search Lab Report

Course Title: Artificial Intelligence & Expert Systems Lab
Course Code: CSE404

Submitted To:

Nahida Marzan
Lecturer, Department of CSE
University of Asia Pacific

Submitted By:

Lamia Tabassum Orpa
Student ID: 22101184

1. Problem Title

Development of a Tic Tac Toe game with an optimal AI player, using the minimax algorithm enhanced with alpha-beta pruning.

2. Problem Statement

Tic Tac Toe is a two-player, zero-sum game played on a 3x3 grid, where each player alternately marks empty cells with their own symbol, X or O. The first player to align three of their own symbols in a row, column, or diagonal wins the game; if all nine cells are filled with no such alignment, the game ends in a draw. Although the rules are simple, building an AI player that always makes the best possible decision is not trivial — the AI has to look ahead at how the game could unfold after each candidate move and choose the one that guarantees the best outcome regardless of how the opponent responds. This is a classic example of an adversarial search problem: two agents with directly opposing goals, one trying to maximize an outcome and the other trying to minimize it, searching through a shared, fully observable game tree.

A brute-force search of the full game tree from an empty board is small enough to be feasible for Tic Tac Toe, but evaluating every single node in that tree is still wasteful, since many branches can be ruled out early without changing the final decision. This assignment addresses both sides of the problem: implementing a correct adversarial search using the minimax algorithm, so that the AI never loses a game it could have won or drawn, and applying alpha-beta pruning on top of it so that branches which cannot affect the outcome are skipped, making the search more efficient without changing its result.

3. Objective

- Implement a fully playable, console-based Tic Tac Toe game on a standard 3x3 board.
- Build an AI opponent that uses the minimax algorithm to always select an optimal move for the current board state.
- Apply alpha-beta pruning on top of minimax to cut down the number of board states evaluated, without changing the move the AI ultimately chooses.
- Support two separate game modes: Human vs Computer and Computer vs Computer, each with an independently adjustable AI search depth.

- Print the board to the console after every move, and clearly report whose turn it is and the final result — a win, a draw, or an ongoing game.

4. Tools and Languages Used

- **Programming Language:** Python 3.13
- **Development Environment:** Visual Studio Code
- **Operating System:** Windows 11
- **Libraries:** None beyond Python's built-in math module, used only for the +infinity / -infinity bounds in alpha-beta pruning. No third-party packages were required.

5. System Design

The board is represented internally as a flat list of 9 cells (indices 0 to 8), which is mapped onto a 3x3 grid purely for display purposes. This keeps win-checking, move validation, and the recursive search simple, since every cell can be addressed directly by a single index rather than a pair of row/column coordinates.

At the top level, the program shows a menu where the user chooses between Human vs Computer and Computer vs Computer, after which the relevant search depth (or depths) are set. From that point on, both modes share the same core loop: print the board, check whether the game has ended, get the next move — either typed in by the human player or computed by the minimax search — apply it to the board, switch the current player, and repeat. Figure 1 below summarizes this control flow.

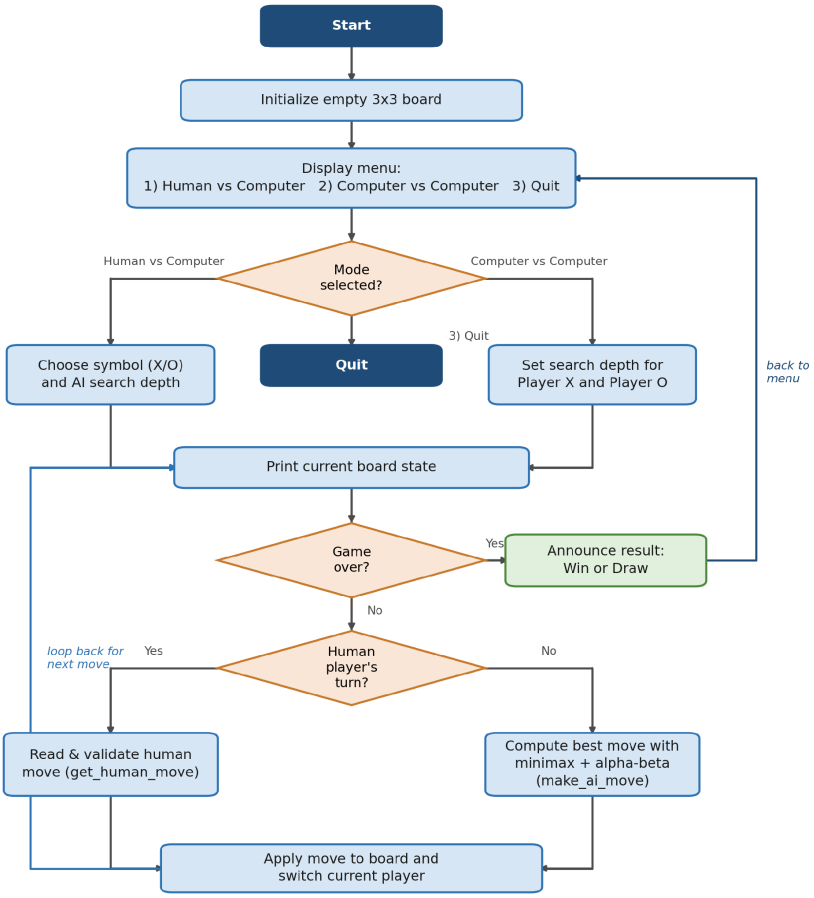

Figure 1: Overall control flow of the Tic Tac Toe program.

6. Algorithm Design

6.1 Board Representation and Win Detection

Eight fixed index combinations cover every possible winning line on a 3x3 board — three rows, three columns, and two diagonals. After every move, the program checks each of these combinations to see if all three cells belong to the same player. The board is considered full, and therefore a draw, once no empty cell index remains. A game is over as soon as either of these two conditions is true.

6.2 Evaluation Function

Since minimax needs a numeric score to compare board states, every terminal (or search-limited) position is scored from the point of view of the AI seat the search is currently being run for:

- +10 — the AI player has won on this board.
- -10 — the opponent has won on this board.

- 0 — the game is a draw, or the search has reached its depth limit without a result.

Keeping the score fixed at +10 / -10 / 0 (rather than, say, rewarding faster wins) is enough for Tic Tac Toe, because the search tree is small enough that the AI always plays out a full game within the chosen depth in practice.

6.3 The Minimax Algorithm

Minimax models the game as two alternating roles. On the AI's turn, the search tries to maximize the score, because the AI wants the best possible outcome for itself. On the opponent's turn, the search tries to minimize that same score, because the opponent is assumed to also play optimally against the AI. The function recurses one ply at a time, trying every open cell, until it hits a finished game, a full board, or the configured depth limit, then backs the resulting scores up the tree so that each level picks the best option available to whichever side is moving at that level. Listing 1 below shows the pseudocode for this recursive process, including the alpha-beta bounds described next.

```
function MINIMAX(board, depth, alpha, beta, maximizingPlayer, player):
    score = EVALUATE(board, player)

    if score == +10 or score == -10:
        return score                        # someone already won
    if board is full or depth == 0:
        return 0                            # draw, or search limit reached

    if maximizingPlayer:
        best = -infinity
        for each empty cell c on board:
            place player's mark at c
            value = MINIMAX(board, depth-1, alpha, beta, False, player)
            undo move at c
            best  = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break                       # beta cut-off
        return best
    else:
        best = +infinity
        for each empty cell c on board:
            place opponent's mark at c
            value = MINIMAX(board, depth-1, alpha, beta, True, player)
            undo move at c
            best = min(best, value)
            beta = min(beta, best)
            if beta <= alpha:
                break                       # alpha cut-off
        return best
```

Listing 1: Pseudocode for the minimax search with alpha-beta pruning.

6.4 Alpha-Beta Pruning

Alpha-beta pruning adds two running bounds to the search: alpha, the best score the maximizing side can already guarantee, and beta, the best score the minimizing side can already guarantee. While exploring a minimizing node, if a child's value drops to or

below alpha, the maximizing side already has a better (or equal) option elsewhere in the tree, so the remaining children of that node cannot change the final decision and are skipped. The symmetric case applies at maximizing nodes against beta. This never changes the value minimax would have returned anyway — it only skips work that could not have affected the result.

Figure 2 walks through a small worked example. The maximizing (AI) node has three minimizing branches. The first branch is evaluated fully and returns 3, which becomes the current alpha. While evaluating the second branch, its first child already returns 2; since 2 is less than or equal to alpha (3), the maximizing side will never choose this branch over the first one no matter what its second child turns out to be, so that second child is pruned without ever being evaluated. The third branch is then evaluated fully and returns 4, which becomes the overall result.

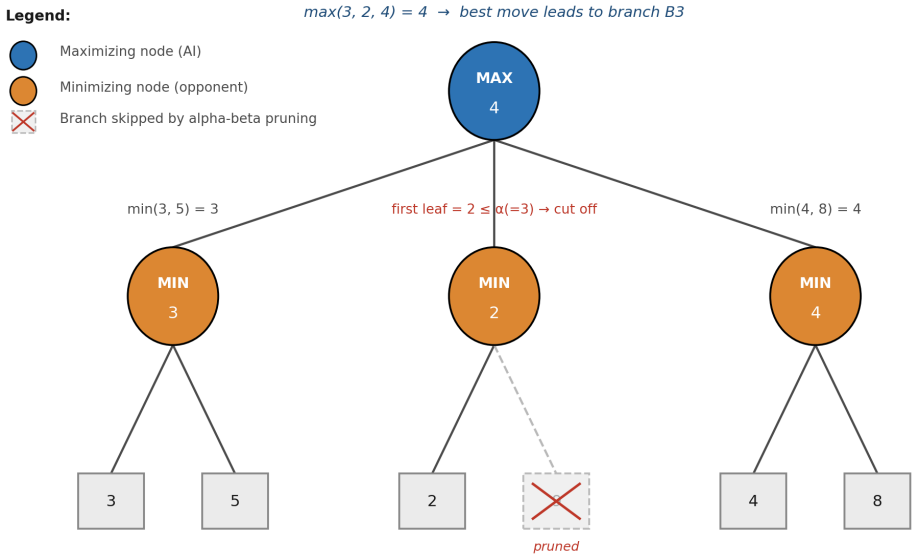

Figure 2: A worked example of alpha-beta pruning cutting off a branch that cannot affect the final decision.

6.5 Depth-Limited Search and Adjustable Difficulty

The implementation also takes a depth parameter, which limits how many plies ahead the search is allowed to look. At depth 9, the AI sees the entire remaining game from any position, which is small enough for Tic Tac Toe to run instantly and guarantees optimal play — the AI will never lose, and will win whenever the opponent allows it. At a lower depth, the AI only looks a few moves ahead, which can make it overlook longer-term threats and lose to a human player. This was added so that both game modes can offer a configurable difficulty rather than only ever playing a perfect, unbeatable game.

7. Implementation Details

The program is organized as a set of small, single-purpose functions rather than one large script, which made the minimax logic much easier to test in isolation. Table 1 summarizes every function in the final program.

Function	Purpose
`print_board(board)`	Prints the current 3x3 board to the console in a readable grid format.
`get_winner(board)`	Checks all 8 winning lines and returns 'X' or 'O' if one of them has won, otherwise None.
`is_full(board)`	Returns True once there are no empty cells left on the board.
`is_game_over(board)`	Combines get_winner and is_full to decide whether the game has ended.
`get_opponent(player)`	Returns the other player's symbol; used throughout the search and the main loop.
`get_empty_cells(board)`	Returns the list of cell indices that are still open for a move.
`evaluate_board(board, player)`	Scores a board from the AI's point of view: +10, -10, or 0 (see Section 6.2).
`minimax(board, depth, alpha, beta, maximizing_player, player)`	The core recursive search described in Section 6.3, with alpha-beta pruning applied.
`make_ai_move(board, player, depth)`	Tries every open cell, asks minimax to score the resulting position, and plays the cell with the highest score.
`get_human_move(board)`	Reads a move from the console and keeps asking until it is a valid, unoccupied cell.
`ask_depth(prompt_text)`	Reads and validates the requested AI search depth (1-9) for a given player.
`announce_result(board, names)`	Prints the final outcome of the game — a named winner, or a draw.
`play_human_vs_computer()`	Runs a full Human vs Computer game: symbol choice, depth choice, and the main turn loop.
`play_computer_vs_computer()`	Runs a full Computer vs Computer game with independently configurable depths for X and O.
`play_game()`	Displays the menu and dispatches to the chosen mode, looping until the user quits.

Table 1: Function reference for tic_tac_toe.py

Both game modes (play_human_vs_computer and play_computer_vs_computer) reuse the exact same minimax and make_ai_move functions; the only difference is where each move comes from on a given turn — console input in the human's case, or a fresh minimax search at that player's configured depth in the computer's case. This sharing of

logic is what guarantees that a 'computer' player behaves identically whether it is playing against a human or against another computer player.

8. Sample Input / Output

The four figures below should be filled in with screenshots captured from the VS Code integrated terminal while running the program. The first two together form one continuous run: Human (X) vs Computer (O) with the AI search depth set to 9, i.e. full-depth, optimal play.

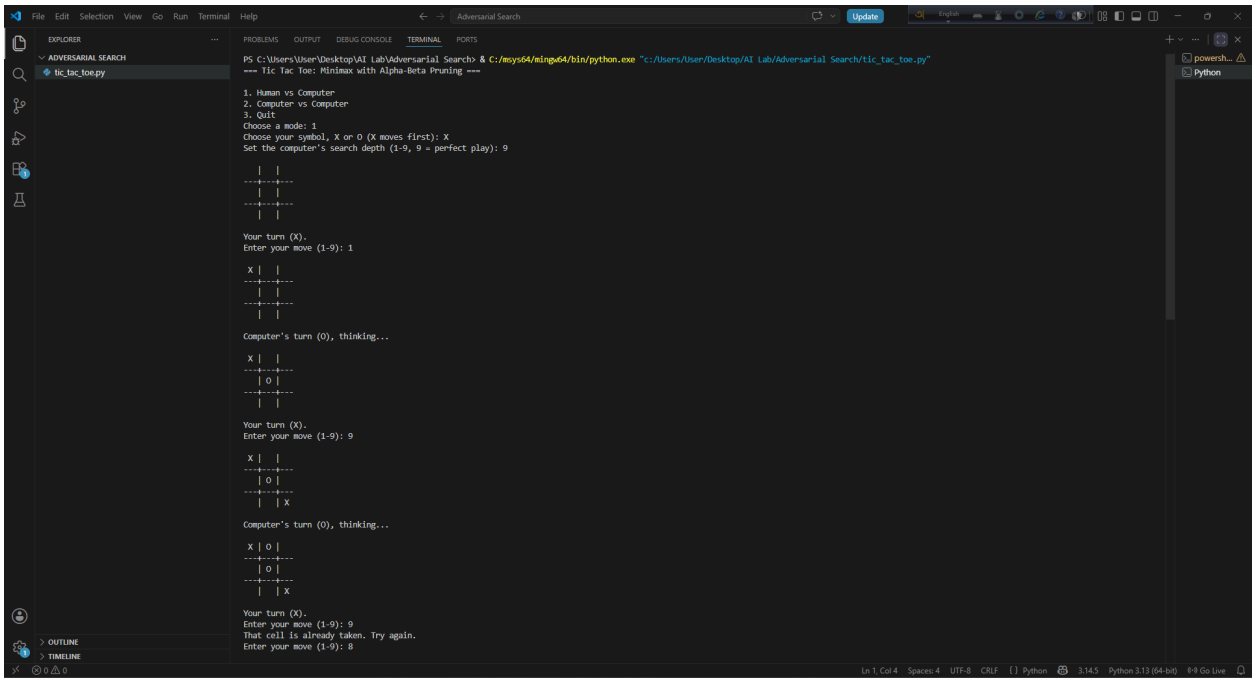

Figure 3: Human vs Computer at depth 9 — mode selection and the opening moves.

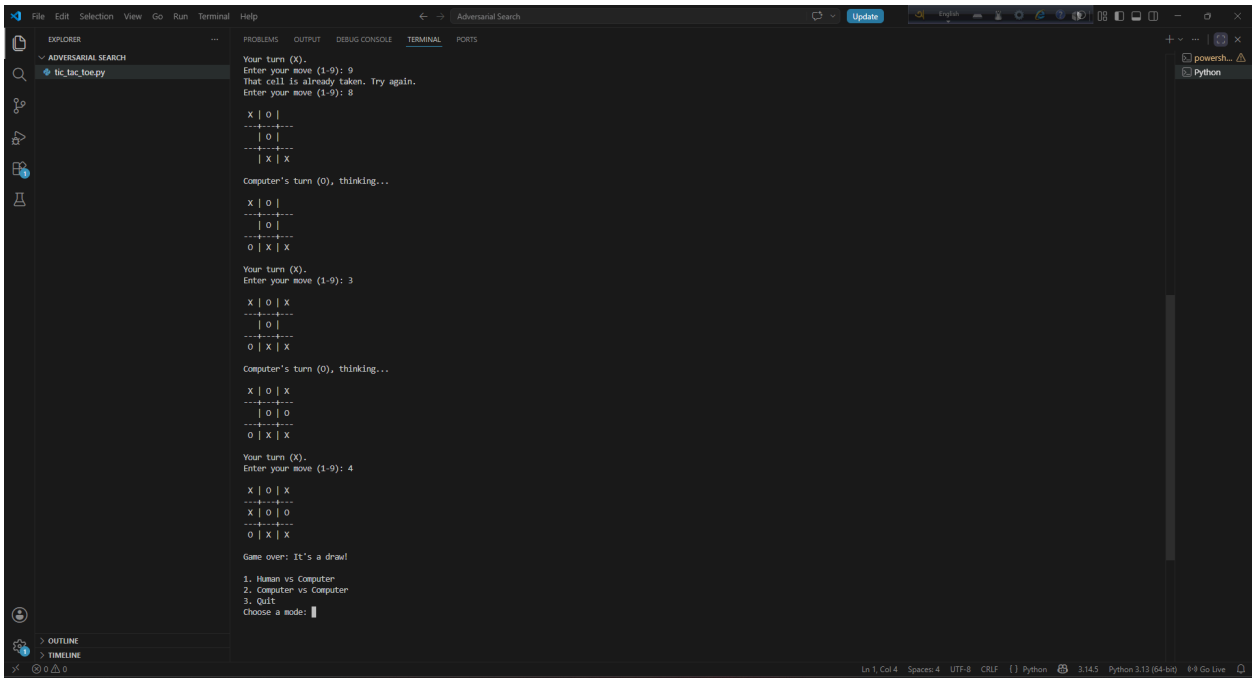

Figure 4: The same game continued — it correctly ends in a draw, exactly as game theory predicts when both sides play optimally.

Two further scenarios are worth demonstrating: a Computer vs Computer match at full depth on both sides (which should also always end in a draw), and a Human vs Computer match where the AI's search depth is deliberately lowered, so the human can win against the now-weaker AI.

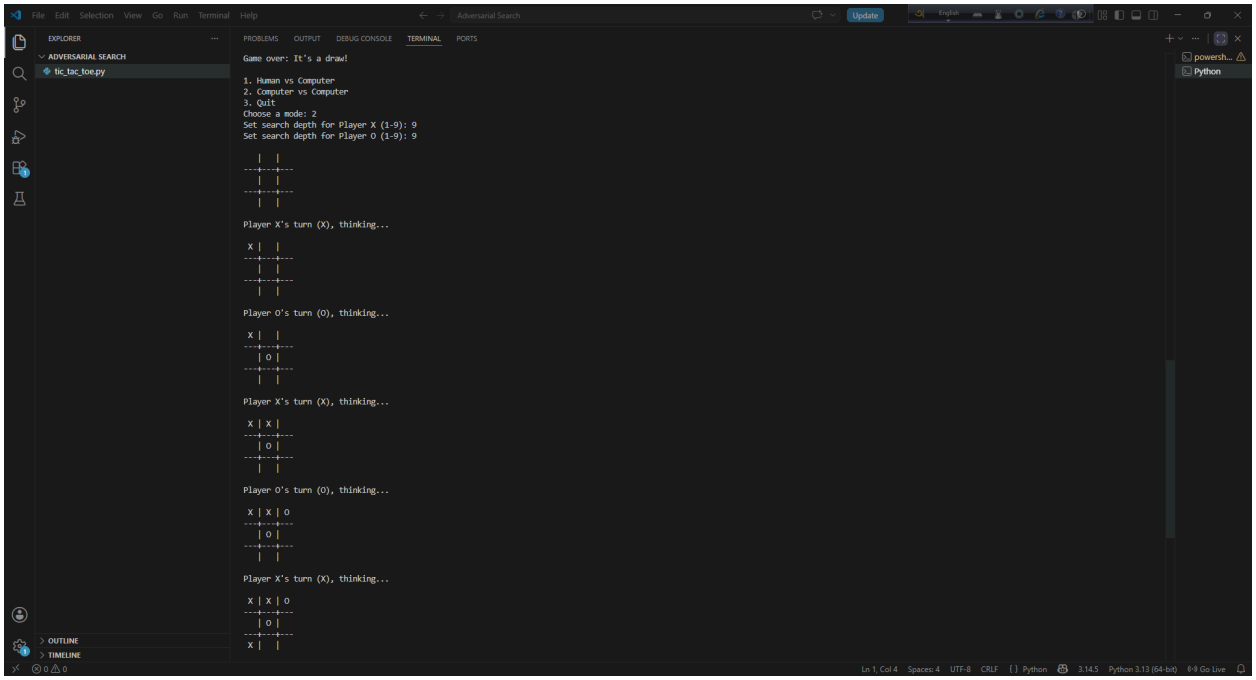

Figure 5: Computer vs Computer at full depth on both sides.

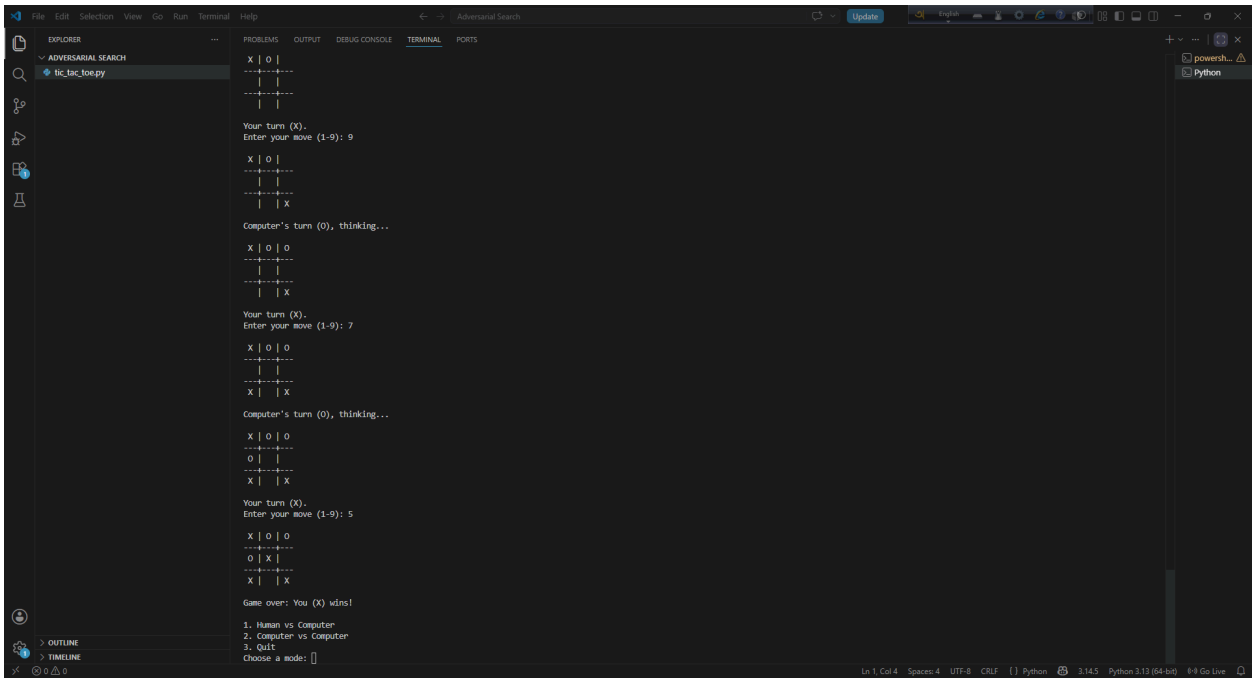

Figure 6: Human defeating a deliberately weakened AI at a lower search depth.

9. Conclusion

This project implements a complete, console-based Tic Tac Toe game in which the computer player decides every move using the minimax algorithm, accelerated with alpha-beta pruning. At full search depth, the AI plays optimally: it never loses, and several test runs of Computer vs Computer at depth 9 confirmed that the game always ends in a draw, exactly as expected for two perfectly played sides in Tic Tac Toe. Lowering the search depth visibly weakens the AI, which makes the Human vs Computer mode usable as an adjustable-difficulty opponent rather than only an unbeatable one.

Working through this assignment made the difference between minimax and alpha-beta pruning concrete rather than just theoretical: both algorithms always arrive at the same move, but alpha-beta consistently explores fewer nodes to get there, especially once a few branches have already established strong alpha or beta bounds. Beyond the algorithm itself, structuring the game into small, independently testable functions made it much easier to confirm correctness, since the recursive search is otherwise hard to reason about by inspection alone.

10. Challenges Faced

Reasoning about maximizing vs minimizing recursion

Keeping track of which side the recursion was scoring for at each level took some getting used to — the same function alternates between trying to maximize and minimize the same evaluation, and it was easy to lose track of whose turn was being simulated several levels deep.

Getting the evaluation function's perspective right

evaluate_board always scores relative to the AI seat the search was originally called for, not whichever side is currently moving inside the recursion. Mixing this up early on produced an AI that occasionally played moves that looked actively self-defeating, which took a while to trace back to the evaluation function rather than the search itself.

Tuning the alpha-beta cut-off condition

The pruning condition (beta <= alpha) needed to be exactly right. A subtly wrong comparison did not break the game outwardly — the AI still appeared to play reasonably — but it either failed to prune anything or, worse, pruned branches it should not have, which is a hard class of bug to notice just by playing the game.

Exposing search depth as a difficulty control

Adding an adjustable depth was straightforward to wire in, but deciding what range made sense (1 to 9) and confirming that a depth-limited search still behaves sensibly — rather than playing an obviously bad move — took some experimentation.

Validating console input

Human input needed to reject non-numeric text, out-of-range numbers, and already-occupied cells without crashing the program, which meant looping on get_human_move until a genuinely legal move was entered.

Convincing myself the search was actually correct

Minimax does not visibly 'look wrong' even when subtly buggy — the most convincing test turned out to be running Computer vs Computer at full depth repeatedly and checking that it always ends in a draw, which is exactly what game theory predicts for optimally played Tic Tac Toe.

11. Appendix: Full Source Code (tic_tac_toe.py)

The complete, runnable source code is reproduced below for reference; it is also submitted separately as tic_tac_toe.py.

```
"""
Tic Tac Toe with Minimax and Alpha-Beta Pruning
-----------------------------------------------
A console-based Tic Tac Toe game where the AI player decides its moves
using the minimax algorithm enhanced with alpha-beta pruning.

Supports:
    1) Human vs Computer  (adjustable AI search depth)
    2) Computer vs Computer (adjustable search depth for both sides)

Scoring used by the AI:
    +10  -> AI wins
    -10  -> Opponent wins
      0  -> Draw / no result yet
"""

import math

EMPTY = ' '
PLAYER_X = 'X'
PLAYER_O = 'O'
BOARD_SIZE = 3

# All winning index combinations for a 3x3 board (rows, columns, diagonals)
WIN_COMBOS = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8),   # rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8),   # columns
    (0, 4, 8), (2, 4, 6),              # diagonals
]


def print_board(board):
    """Prints the 3x3 board to the console in a readable grid form."""
    print()
    for row in range(3):
        cells = board[row * 3: row * 3 + 3]
        line = " " + " | ".join(c if c != EMPTY else " " for c in cells)
        print(line)
        if row < 2:
            print("---+---+---")
    print()


def get_winner(board):
    """Returns 'X' or 'O' if there is a winner on the board, else None."""
    for a, b, c in WIN_COMBOS:
        if board[a] != EMPTY and board[a] == board[b] == board[c]:
            return board[a]
    return None


def is_full(board):
    """True if there are no empty cells left."""
    return EMPTY not in board


def is_game_over(board):
    """Game ends when someone has won or the board is full (draw)."""
    return get_winner(board) is not None or is_full(board)


def get_opponent(player):
    """Returns the other player's symbol."""
    return PLAYER_O if player == PLAYER_X else PLAYER_X
```

```
def get_empty_cells(board):
    """Returns a list of indices of all empty cells."""
    return [i for i, val in enumerate(board) if val == EMPTY]


def evaluate_board(board, player):
    """
    Scores a terminal/near-terminal board from the perspective of `player`
    (the AI seat we are computing a move for).
        +10 if `player` has won
        -10 if the opponent has won
          0 otherwise (draw or non-terminal)
    """
    winner = get_winner(board)
    if winner == player:
        return 10
    if winner == get_opponent(player):
        return -10
    return 0


def minimax(board, depth, alpha, beta, maximizing_player, player):
    """
    Minimax with alpha-beta pruning.

    board             -> current board state
    depth             -> remaining search depth (limits how far ahead the AI looks)
    alpha, beta       -> pruning bounds
    maximizing_player -> True if it's `player`'s (the AI's) turn to move here,
                         False if it's the opponent's turn
    player            -> the symbol the AI is computing the best move for
    """
    score = evaluate_board(board, player)

    # Terminal states: someone already won
    if score == 10 or score == -10:
        return score

    # Draw, or we've run out of search depth
    if is_full(board) or depth == 0:
        return 0

    if maximizing_player:
        best = -math.inf
        for cell in get_empty_cells(board):
            board[cell] = player
            value = minimax(board, depth - 1, alpha, beta, False, player)
            board[cell] = EMPTY
            best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break  # beta cut-off
        return best
    else:
        best = math.inf
        opponent = get_opponent(player)
        for cell in get_empty_cells(board):
            board[cell] = opponent
            value = minimax(board, depth - 1, alpha, beta, True, player)
            board[cell] = EMPTY
            best = min(best, value)
            beta = min(beta, best)
            if beta <= alpha:
                break  # alpha cut-off
        return best


def make_ai_move(board, player, depth):
    """
    Picks and plays the best move for `player` on `board`, searching up to
    `depth` plies ahead using minimax + alpha-beta pruning.
    Returns the index of the cell that was played (or None if no move was possible).
    """
    best_score = -math.inf
```

```
    best_move = None

    for cell in get_empty_cells(board):
        board[cell] = player
        score = minimax(board, depth - 1, -math.inf, math.inf, False, player)
        board[cell] = EMPTY

        if score > best_score:
            best_score = score
            best_move = cell

    if best_move is not None:
        board[best_move] = player

    return best_move


def get_human_move(board):
    """Prompts the human for a move (1-9, left-to-right/top-to-bottom) and validates it."""
    while True:
        raw = input("Enter your move (1-9): ").strip()
        if not raw.isdigit():
            print("Please enter a number between 1 and 9.")
            continue
        move = int(raw) - 1
        if move < 0 or move > 8:
            print("Please enter a number between 1 and 9.")
            continue
        if board[move] != EMPTY:
            print("That cell is already taken. Try again.")
            continue
        return move


def ask_depth(prompt_text):
    """Asks for a search depth between 1 and 9, defaulting to 9 (perfect play) on bad
input."""
    raw = input(prompt_text).strip()
    if raw.isdigit() and 1 <= int(raw) <= 9:
        return int(raw)
    print("Invalid input, defaulting to depth 9 (optimal play).")
    return 9


def announce_result(board, names):
    """Prints the final result of the game using the provided name mapping."""
    winner = get_winner(board)
    if winner:
        print(f"Game over: {names[winner]} ({winner}) wins!")
    else:
        print("Game over: It's a draw!")


def play_human_vs_computer():
    board = [EMPTY] * 9

    human = input("Choose your symbol, X or O (X moves first): ").strip().upper()
    while human not in (PLAYER_X, PLAYER_O):
        human = input("Please type X or O: ").strip().upper()
    computer = get_opponent(human)

    depth = ask_depth("Set the computer's search depth (1-9, 9 = perfect play): ")

    names = {human: "You", computer: "Computer"}
    current = PLAYER_X
    print_board(board)

    while not is_game_over(board):
        if current == human:
            print(f"Your turn ({human}).")
            move = get_human_move(board)
            board[move] = human
        else:
            print(f"Computer's turn ({computer}), thinking...")
```

```
            make_ai_move(board, computer, depth)
        print_board(board)
        current = get_opponent(current)

    announce_result(board, names)


def play_computer_vs_computer():
    board = [EMPTY] * 9

    depth_x = ask_depth("Set search depth for Player X (1-9): ")
    depth_o = ask_depth("Set search depth for Player O (1-9): ")

    names = {PLAYER_X: "Player X", PLAYER_O: "Player O"}
    current = PLAYER_X
    print_board(board)

    while not is_game_over(board):
        depth = depth_x if current == PLAYER_X else depth_o
        print(f"{names[current]}'s turn ({current}), thinking...")
        make_ai_move(board, current, depth)
        print_board(board)
        current = get_opponent(current)

    announce_result(board, names)


def play_game():
    print("=== Tic Tac Toe: Minimax with Alpha-Beta Pruning ===")
    while True:
        print("\n1. Human vs Computer")
        print("2. Computer vs Computer")
        print("3. Quit")
        choice = input("Choose a mode: ").strip()

        if choice == '1':
            play_human_vs_computer()
        elif choice == '2':
            play_computer_vs_computer()
        elif choice == '3':
            print("Thanks for playing!")
            break
        else:
            print("Invalid choice, please enter 1, 2, or 3.")


if __name__ == "__main__":
    play_game()
```